

Lab 7 - DLX Hazards

Nate Herbst

A02307138

Nathan Walker

A02364124

Introduction

In a pipelined processor, multiple instructions execute simultaneously across different pipeline stages. While pipelining increases throughput, it introduces hazards — situations where the next instruction cannot execute correctly in the following clock cycle due to dependencies on previous instructions still in the pipeline. In the previous labs (Labs 3–6), these hazards were avoided by manually inserting NOP instructions into the assembly code, which is impractical for real software. Lab 7 requires the processor hardware itself to detect and resolve these hazards automatically.

This lab addresses two categories of hazards in the custom 5-stage DLX pipeline (Fetch, Decode, Execute, Memory, Write-Back):

1. **Data hazards** — specifically Read-After-Write (RAW) dependencies, where an instruction needs a register value that a preceding instruction has not yet written back. These are resolved through data forwarding (bypassing results from later pipeline stages back to the Execute stage inputs) and pipeline stalling for the special case of load-use hazards, where a load word (LW) result is not available until after the Memory stage and therefore cannot be forwarded in time.
2. **Control hazards** — caused by branch and jump instructions that redirect the program counter. Instructions fetched after a branch but before the branch decision is resolved are invalid and must be squashed. This is handled by flushing the pipeline (replacing the incorrectly fetched instructions with NOPs).

The goal is to execute the factorial program — which computes $\text{factorial}(6) = 720$ — without any NOP instructions, with all hazards resolved by the hardware.

Procedure

Modular Systems

Three new hardware modules were created, and three existing modules were modified. All design was done in structural/behavioral VHDL.

New Module: Forwarding Unit (forwarding_unit.vhd)

This purely combinational module detects RAW data hazards by comparing the source register addresses of the instruction currently in the Execute stage (ID/EX) against the destination register addresses of instructions in the Memory stage (EX/MEM) and the Write-Back stage (MEM/WB). It outputs two 2-bit MUX select signals (fwd_a_sel for the rs1 ALU input, fwd_b_sel for the rs2 ALU input) with the following encoding:

- "00" — no forwarding; use the register file output
- "01" — forward from EX/MEM (the ALU result currently in the Memory stage)
- "10" — forward from MEM/WB (the write-back data from the Write-Back stage)

EX/MEM forwarding has higher priority than MEM/WB forwarding, since it represents a more recent result. Forwarding to or from R0 is suppressed since R0 is hardwired to zero.

New Module: Hazard Detection Unit (hazard_detection.vhd)

This combinational module detects two hazard conditions:

Load-use stall: When the instruction in Execute is a LW whose destination register matches a source register of the instruction in Decode, a one-cycle stall is asserted. This freezes the Fetch stage (PC holds its value) and injects a NOP bubble into the Execute stage. After the stall cycle, the loaded value is available for MEM/WB forwarding.

Control hazard flush: When branch_en is asserted by the Execute stage, the flush signal is raised. In the top-level processor, this flush is extended to two cycles using a delay register, since the branch decision arrives two cycles after the branch instruction was fetched (due to the ROM's internal address latch and the pipeline depth). Flush overrides stall.

Modified Module: Decode Stage (decode.vhd)

The decode stage was modified to support stall and flush inputs. When either signal is asserted, a bubble signal drives decode_rst, which resets all ID/EX pipeline registers to zero (injecting a NOP). A critical design requirement discovered during development was that the register file's read data outputs (rs1_data, rs2_data) must pass through pipeline registers (using the same reggi component as the instruction and immediate fields). Without these registers, the register file outputs are combinational — they change as soon as the next instruction arrives at instruction_in, causing the data to be misaligned with the registered instruction by one cycle.

Modified Module: Execute Stage (execute.vhd)

Two 3-to-1 forwarding MUXes were added at the ALU inputs, controlled by fwd_a_sel and fwd_b_sel. These select between the register file data (default), the EX/MEM ALU result, or the MEM/WB write-back data. A flush_rst signal (the OR of rst and flush) was added to all output pipeline registers so that when a flush occurs, the Execute stage outputs are cleared to NOP.

Modified Module: Fetch Stage (fetch.vhd)

Stall support was added via MUXes on the PC register and decode address register inputs. When stall = '1', these MUXes feed back the current values rather than the incremented PC, effectively freezing the Fetch stage for one cycle.

Top-Level Processor (DLX_Processor.vhd)

The top-level module was rewritten to instantiate and wire together all five pipeline stages plus the forwarding unit and hazard detection unit. Key wiring includes:

A flush delay register that extends the branch flush to two cycles (flush = flush_raw OR flush_r1)
Combinational extraction of source and destination register addresses from the pipeline instruction registers, matching the decode stage's extraction logic for the custom DLX opcode encoding
Gating of the stall signal by flush (stall = stall_raw AND NOT flush) to prevent stalling during a flush

Challenges and Solutions

- **Pipeline Alignment Bug in Decode:** The most significant challenge was discovering that the register file data outputs were misaligned with the instruction pipeline register. In the decode stage, instruction_out, sign_ext_imm, and pc_inc_out all pass through reggi pipeline registers and update on the clock edge. However, rs1_data and rs2_data were originally connected directly from the register file's combinational read ports to the entity output. After each clock edge, instruction_in changes to the next instruction, which immediately changes the address inputs to the register file, causing rs1_data/rs2_data to reflect the next instruction's operands — not the current one being sent to Execute via instruction_out. This one-cycle misalignment meant the Execute stage received the instruction word for instruction I but register data for instruction I+1. The fix was to add reggi pipeline registers to rs1_data, rs2_data, and rd_addr_out, all clocked with decode_rst so they properly insert NOP bubbles on stall or flush.

- **Two-Cycle Branch Penalty:** The branch/jump decision is made in the Execute stage, but because the Fetch stage's ROM uses a synchronous address register (the altsyncram's internal clock input), there is effectively one extra cycle of latency. A single-cycle flush was insufficient to squash all incorrectly fetched instructions. A delay register was added in the top-level processor to extend the flush signal to two cycles.
- **Forwarding Priority:** When two pipeline stages both produce a result for the same destination register (e.g., two consecutive ADD instructions writing to R5), the forwarding unit must select the most recent value. This was handled by checking MEM/WB first (lower priority) and then EX/MEM (higher priority), so the EX/MEM match overwrites the MEM/WB match in the sequential sensitivity process.

Results

The DLX processor with hazard mitigation was simulated in Questa using the factorial program without any NOP instructions. The program loads the value 6 from data memory address 1, computes factorial(6) through an iterative multiply-by-addition loop using branches (BEQZ, BNEZ), jumps (J, JAL, JR), and arithmetic instructions (ADD, SUBI, ADDI), then stores the result to data memory address 0. The forwarding unit correctly bypassed ALU results and write-back data to avoid stalls on back-to-back dependent instructions. The hazard detection unit inserted stalls for load-use cases and flushed the pipeline on all taken branches and jumps. The expected result of 720 (0x2D0) was written to data memory address 0, confirming correct operation.

Figures

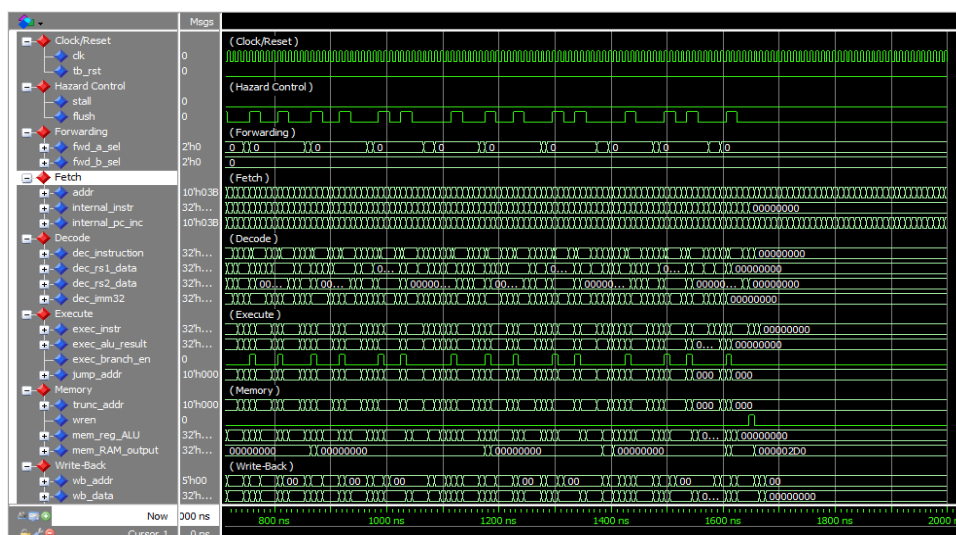


Figure 2: simulation showing memory is 2D0 at the end

Conclusion

This lab demonstrated the hardware implementation of pipeline hazard mitigation in a 5-stage DLX processor. Data forwarding eliminates the majority of RAW data hazard stalls by bypassing results directly from the Memory and Write-Back stages to the Execute stage. Load-use hazards, where forwarding alone is insufficient, are handled by a single-cycle pipeline stall followed by MEM/WB forwarding. Control hazards from branches and jumps are handled by flushing the pipeline for two cycles when a branch is taken. Together, these mechanisms allow the processor to execute programs without programmer-inserted NOP instructions, correctly computing $\text{factorial}(6) = 720$ as verified in simulation.

Appendix

DLX_Processor.vhd (top level file)

```
library ieee;
use ieee.std_logic_1164.all;

library work;
use work.fetch_pkg.all;
use work.decode_reg_pkg.all;

entity DLX_Processor is
    generic (
        WIDTH          : integer := 10;
        INSTR_WIDTH    : integer := 32
    );
    port (
        clk             : in  std_logic;
        rst             : in  std_logic
    );
end entity DLX_Processor;

architecture structural of DLX_Processor is
    -- Fetch -> Decode signals
    signal internal_pc_inc : std_logic_vector(WIDTH-1 downto 0);
    signal internal_instr  : std_logic_vector(INSTR_WIDTH-1 downto 0);

    -- Decode -> Execute signals
    signal dec_instruction : std_logic_vector(INSTR_WIDTH-1 downto 0);
    signal dec_rs1_data    : std_logic_vector(INSTR_WIDTH-1 downto 0);
    signal dec_rs2_data    : std_logic_vector(INSTR_WIDTH-1 downto 0);
    signal dec_imm32       : std_logic_vector(INSTR_WIDTH-1 downto 0);
    signal dec_rd_addr     : std_logic_vector(4 downto 0);
    signal dec_pc_inc      : std_logic_vector(WIDTH-1 downto 0);

    -- Execute outputs / feedback
    signal exec_branch_en  : std_logic;
    signal exec_alu_result : std_logic_vector(INSTR_WIDTH-1 downto 0);
    signal exec_rs2_data   : std_logic_vector(INSTR_WIDTH-1 downto 0);
```

```

signal exec_instr      : std_logic_vector(INSTR_WIDTH-1 downto 0);
signal exec_rd_addr    : std_logic_vector(4 downto 0);
signal exec_pc_out     : std_logic_vector(9 downto 0);

-- Jump address derived from Execute
signal jump_addr       : std_logic_vector(WIDTH-1 downto 0);

-- Memory stage signals
signal mem_RAM_output  : std_logic_vector(INSTR_WIDTH-1 downto 0);
signal mem_reg_ALU     : std_logic_vector(INSTR_WIDTH-1 downto 0);
signal mem_instr_out   : std_logic_vector(INSTR_WIDTH-1 downto 0);
signal mem_pc_out      : std_logic_vector(9 downto 0);

-- Write-back signals
signal wb_en           : std_logic;
signal wb_data         : std_logic_vector(INSTR_WIDTH-1 downto 0);
signal wb_addr         : std_logic_vector(4 downto 0);

-----

-- HAZARD CONTROL SIGNALS

-----

-- Stall (load-use hazard)
signal stall_raw       : std_logic;
signal stall           : std_logic;

-- Flush (control hazard - branch/jump taken)
signal flush_raw       : std_logic;
signal flush_r1        : std_logic := '0'; -- 1-cycle delayed flush
signal flush           : std_logic;

-----

-- FORWARDING SIGNALS

```

```

-----
-
signal fwd_a_sel      : std_logic_vector(1 downto 0);
signal fwd_b_sel      : std_logic_vector(1 downto 0);

-- Source register addresses of instruction in Execute (ID/EX)
signal id_ex_opcode    : std_logic_vector(5 downto 0);
signal id_ex_rs1_addr  : std_logic_vector(4 downto 0);
signal id_ex_rs2_addr  : std_logic_vector(4 downto 0);

-- Destination info from instruction in Memory (EX/MEM)
signal ex_mem_opcode   : std_logic_vector(5 downto 0);
signal ex_mem_rd_addr  : std_logic_vector(4 downto 0);
signal ex_mem_wb_en    : std_logic;

begin

-----
-

-- FLUSH DELAY REGISTER
-- Branch penalty is 2 cycles (due to ROM address register + pipeline).
-- flush = branch_en OR delayed_branch_en

-----
-

process(clk)
begin
    if rising_edge(clk) then
        if rst = '1' then
            flush_r1 <= '0';
        else
            flush_r1 <= flush_raw;
        end if;
    end if;
end process;

flush_raw <= exec_branch_en;
flush     <= flush_raw or flush_r1;

```

```

-- Stall gated by flush (no stall during flush)
stall <= stall_raw and not flush;

-----
-
-- EXTRACT SOURCE REGISTER ADDRESSES FROM ID/EX INSTRUCTION
-- (matches decode.vhd extraction logic)
-----
-

id_ex_opcode <= dec_instruction(31 downto 26);

-- rs1: [25:21] for BEQZ/BNEZ (register to test), else [20:16]
id_ex_rs1_addr <= dec_instruction(25 downto 21)
                 when (id_ex_opcode = OP_BEQZ or id_ex_opcode =
OP_BNEZ)
                 else dec_instruction(20 downto 16);

-- rs2: [25:21] for SW/JR/JALR, else [15:11]
id_ex_rs2_addr <= dec_instruction(25 downto 21)
                 when (id_ex_opcode = OP_SW or
id_ex_opcode = OP_JR or
id_ex_opcode = OP_JALR)
                 else dec_instruction(15 downto 11);

-----
-
-- EXTRACT DESTINATION INFO FROM EX/MEM INSTRUCTION
-- (matches write_back.vhd logic)
-----
-

ex_mem_opcode <= exec_instr(31 downto 26);

-- Destination register: R31 for JAL/JALR, else [25:21]
ex_mem_rd_addr <= "11111"

```

```
when (ex_mem_opcode = OP_JAL or ex_mem_opcode =  
OP_JALR)
```

```
else exec_instr(25 downto 21);
```

```
-- Write-enable: '0' for NOP, SW, BEQZ, BNEZ, J, JR
```

```
ex_mem_wb_en <= '0' when (ex_mem_opcode = OP_NOP or  
ex_mem_opcode = OP_SW or  
ex_mem_opcode = OP_BEQZ or  
ex_mem_opcode = OP_BNEZ or  
ex_mem_opcode = OP_J or  
ex_mem_opcode = OP_JR)
```

```
else '1';
```

```
-- FORWARDING UNIT
```

```
fwd_unit: entity work.forwarding_unit
```

```
port map (  
    id_ex_rs1_addr => id_ex_rs1_addr,  
    id_ex_rs2_addr => id_ex_rs2_addr,  
    ex_mem_rd_addr => ex_mem_rd_addr,  
    ex_mem_wb_en   => ex_mem_wb_en,  
    mem_wb_rd_addr => wb_addr,  
    mem_wb_wb_en   => wb_en,  
    fwd_a_sel      => fwd_a_sel,  
    fwd_b_sel      => fwd_b_sel  
);
```

```
-- HAZARD DETECTION UNIT
```

```
hazard_unit: entity work.hazard_detection
```

```

    port map (
        id_ex_instruction => dec_instruction,
        if_id_instruction => internal_instr,
        branch_en         => exec_branch_en,
        stall              => stall_raw,
        flush              => open  -- we compute flush separately with
delay
    );

```

```

-- FETCH STAGE

```

```

fetch_inst: fetch
    generic map (
        N => WIDTH,
        M => INSTR_WIDTH
    )
    port map (
        jump_addr    => jump_addr,
        pc_select    => exec_branch_en,
        stall        => stall,
        rst          => rst,
        clk          => clk,
        decode_addr  => internal_pc_inc,
        instruction  => internal_instr
    );

```

```

-- DECODE STAGE

```

```

decode_inst: entity work.decode
    generic map (

```

```

        FUNC_WIDTH => 6,
        ADDR_WIDTH => 5,
        DATA_WIDTH => 32
    )
    port map (
        clk           => clk,
        rst           => rst,
        stall         => stall,
        flush         => flush,
        instruction_in => internal_instr,
        pc_inc        => internal_pc_inc,
        wb_data       => wb_data,
        wb_addr       => wb_addr,
        wb_en         => wb_en,
        rs1_data      => dec_rs1_data,
        rs2_data      => dec_rs2_data,
        sign_ext_imm  => dec_imm32,
        rd_addr_out   => dec_rd_addr,
        pc_inc_out    => dec_pc_inc,
        instruction_out => dec_instruction
    );

```

```

-- EXECUTE STAGE

```

```

execute_inst: entity work.execute
    generic map (
        DATA_WIDTH => INSTR_WIDTH,
        PC_WIDTH    => WIDTH
    )
    port map (
        clk           => clk,
        rst           => rst,
        instruction    => dec_instruction,
        pc_inc        => dec_pc_inc,
        rs1_data      => dec_rs1_data,

```

```

    rs2_data      => dec_rs2_data,
    sign_ext_imm  => dec_imm32,
    rd_addr_in    => dec_rd_addr,
    -- Hazard control
    flush         => flush,
    -- Forwarding
    fwd_a_sel     => fwd_a_sel,
    fwd_b_sel     => fwd_b_sel,
    ex_mem_alu_data => exec_alu_result,
    mem_wb_data   => wb_data,
    -- Outputs
    Branch_en     => exec_branch_en,
    ALU_result    => exec_alu_result,
    rs2_data_out  => exec_rs2_data,
    instr_out     => exec_instr,
    rd_addr_out   => exec_rd_addr,
    pc_out        => exec_pc_out,
    jump_addr     => jump_addr
);

```

```

-- MEMORY STAGE

```

```

memory_inst: entity work.memory
generic map(
    DATA_WIDTH => INSTR_WIDTH
)
port map(
    clk         => clk,
    rst         => rst,
    pc_inc      => exec_pc_out,
    instruction => exec_instr,
    ALU_result  => exec_alu_result,
    rs2_data    => exec_rs2_data,
    RAM_output  => mem_RAM_output,
    reg_ALU     => mem_reg_ALU,

```

```

        instr_out    => mem_instr_out,
        pc_out       => mem_pc_out
    );

--
-- WRITE-BACK STAGE
--

write_back_inst: entity work.write_back
    generic map(
        DATA_WIDTH => INSTR_WIDTH
    )
    port map(
        RAM_output    => mem_RAM_output,
        reg_ALU        => mem_reg_ALU,
        pc_inc         => mem_pc_out,
        instruction    => mem_instr_out,
        wb_en          => wb_en,
        wb_data        => wb_data,
        wb_addr        => wb_addr
    );

end architecture structural;

```

hazard_detection.vhd

```

library ieee;
use ieee.std_logic_1164.all;

library work;
use work.decode_reg_pkg.all;

-- Hazard Detection Unit
--
-- Detects two types of pipeline hazards:

```

```

--
-- 1. LOAD-USE HAZARD (stall):
--   A LW in Execute (ID/EX) writes to a register that the instruction
--   in Decode (IF/ID) needs to read. Even forwarding can't help because
--   the load data won't be available until after the Memory stage.
--   Solution: stall for 1 cycle, then forward from MEM/WB.
--
-- 2. CONTROL HAZARD (flush):
--   A branch/jump is taken. The 2 instructions that entered the pipeline
--   behind the branch are wrong and must be squashed (replaced with
NOPs).
--
-- When stall = '1':
--   - Fetch freezes (PC and IF/ID registers don't update)
--   - Decode outputs NOP to Execute (bubble inserted)
--   - The LW proceeds through Execute → Memory normally
--
-- When flush = '1':
--   - Decode outputs NOP to Execute (squash wrong instruction)
--   - Execute output registers latch NOP (squash wrong instruction)
--   - Fetch PC is already redirected by branch_en/jump_addr
--
-- Flush takes priority over stall.

entity hazard_detection is
    port (
        -- Instruction entering Execute (from decode's registered output,
ID/EX)
        id_ex_instruction : in  std_logic_vector(31 downto 0);

        -- Instruction entering Decode (from fetch output, IF/ID)
        if_id_instruction : in  std_logic_vector(31 downto 0);

        -- Branch/jump taken signal from Execute stage output register
        branch_en          : in  std_logic;

        -- Stall: freeze Fetch, inject NOP bubble into Execute
        stall              : out std_logic;

        -- Flush: inject NOP at Decode output and Execute output

```

```

        flush            : out std_logic
    );
end entity hazard_detection;

architecture behavioral of hazard_detection is

    -- ID/EX (Execute) instruction fields
    signal id_ex_opcode : std_logic_vector(5 downto 0);
    signal id_ex_rd      : std_logic_vector(4 downto 0);

    -- IF/ID (Decode) instruction fields
    signal if_id_opcode : std_logic_vector(5 downto 0);
    signal if_id_rd      : std_logic_vector(4 downto 0);
    signal if_id_rs1     : std_logic_vector(4 downto 0);
    signal if_id_rs2     : std_logic_vector(4 downto 0);

    -- Does the IF/ID instruction actually read these source registers?
    signal uses_rs1      : std_logic;
    signal uses_rs2      : std_logic;

    -- Internal hazard flag
    signal load_use      : std_logic;

begin

-----
--
    -- Field extraction
-----
--

    id_ex_opcode <= id_ex_instruction(31 downto 26);
    id_ex_rd     <= id_ex_instruction(25 downto 21);

    if_id_opcode <= if_id_instruction(31 downto 26);
    if_id_rd     <= if_id_instruction(25 downto 21);
    -- IF/ID rs1 address (same extraction logic as decode.vhd)
    --   BEQZ/BNEZ: register to test is at bits [25:21]
    --   All others: rs1 is at bits [20:16]

```

```

if_id_rs1 <= if_id_instruction(25 downto 21)
    when (if_id_opcode = OP_BEQZ or if_id_opcode = OP_BNEZ)
    else if_id_instruction(20 downto 16);

-- IF/ID rs2 address (same extraction logic as decode.vhd)
-- SW/JR/JALR: second source register is at bits [25:21]
-- R-type:      rs2 is at bits [15:11]
if_id_rs2 <= if_id_instruction(25 downto 21)
    when (if_id_opcode = OP_SW or
          if_id_opcode = OP_JR or
          if_id_opcode = OP_JALR)
    else if_id_instruction(15 downto 11);

-----
--
-- Determine which source registers are actually used by IF/ID
instruction
-----
--
-- rs1 is used by everything EXCEPT: NOP, J, JAL, JR, JALR
uses_rs1 <= '0' when (if_id_opcode = OP_NOP or
                    if_id_opcode = OP_J or
                    if_id_opcode = OP_JAL or
                    if_id_opcode = OP_JR or
                    if_id_opcode = OP_JALR)
                    else '1';

-- rs2 is used only by: R-type ALU ops, SW, JR, JALR
uses_rs2 <= '1' when (if_id_opcode = OP_ADD or if_id_opcode = OP_ADDU
or
                    if_id_opcode = OP_SUB or if_id_opcode = OP_SUBU
or
                    if_id_opcode = OP_AND or if_id_opcode = OP_OR
or
                    if_id_opcode = OP_XOR or
                    if_id_opcode = OP_SLL or if_id_opcode = OP_SRL
or

```

```

        if_id_opcode = OP_SRA or
        if_id_opcode = OP_SLT or if_id_opcode = OP_SLTU
or
        if_id_opcode = OP_SGT or if_id_opcode = OP_SGTU
or
        if_id_opcode = OP_SLE or if_id_opcode = OP_SLEU
or
        if_id_opcode = OP_SGE or if_id_opcode = OP_SGEU
or
        if_id_opcode = OP_SEQ or if_id_opcode = OP_SNE
or
        if_id_opcode = OP_SW or
        if_id_opcode = OP_JR or if_id_opcode = OP_JALR)
    else '0';

```

```

-- Load-use hazard detection

```

```

-- Stall required when:
-- 1. Instruction in Execute is LW
-- 2. LW destination is not R0
-- 3. IF/ID instruction reads from the LW destination

```

```

load_use <= '1' when (id_ex_opcode = OP_LW and
                    id_ex_rd /= "00000" and
                    ((uses_rs1 = '1' and if_id_rs1 = id_ex_rd) or
                     (uses_rs2 = '1' and if_id_rs2 = id_ex_rd)))
    else '0';

```

```

-- Output signals

```

```

-- Flush overrides stall (branch squashes everything)

```

```

    stall <= load_use and not branch_en;
    flush <= branch_en;

end architecture behavioral;

```

forwarding_unit.vhd

```

library ieee;
use ieee.std_logic_1164.all;

-- Forwarding Unit
-- Detects RAW (read-after-write) data hazards and selects the correct
-- forwarding source for the Execute stage ALU inputs.
--
-- Forwarding MUX encoding:
--   "00" = no forwarding, use register file output
--   "01" = forward from EX/MEM (ALU result from Memory stage)
--   "10" = forward from MEM/WB (write-back data from WriteBack stage)
--
-- Priority: EX/MEM > MEM/WB (more recent result wins)
-- Never forwards to/from R0 (hardwired zero)

entity forwarding_unit is
    port (
        -- Source register addresses of instruction in Execute (ID/EX)
        id_ex_rs1_addr    : in  std_logic_vector(4 downto 0);
        id_ex_rs2_addr    : in  std_logic_vector(4 downto 0);

        -- Destination info from instruction in Memory (EX/MEM)
        ex_mem_rd_addr    : in  std_logic_vector(4 downto 0);
        ex_mem_wb_en      : in  std_logic;

        -- Destination info from instruction in WriteBack (MEM/WB)
        mem_wb_rd_addr    : in  std_logic_vector(4 downto 0);
        mem_wb_wb_en      : in  std_logic;

        -- Forwarding MUX selects
        fwd_a_sel          : out std_logic_vector(1 downto 0); -- rs1 path
    );
end entity forwarding_unit;

```

```

        fwd_b_sel          : out std_logic_vector(1 downto 0)  -- rs2 path
    );
end entity forwarding_unit;

architecture behavioral of forwarding_unit is
begin

    fwd_proc : process(id_ex_rs1_addr, id_ex_rs2_addr,
                      ex_mem_rd_addr, ex_mem_wb_en,
                      mem_wb_rd_addr, mem_wb_wb_en)
    begin
        -- Default: no forwarding
        fwd_a_sel <= "00";
        fwd_b_sel <= "00";

        -----
        -- MEM/WB forwarding (lower priority - checked first so that
        -- the EX/MEM check below can override it)
        -----

        if mem_wb_wb_en = '1' and mem_wb_rd_addr /= "00000" then
            if mem_wb_rd_addr = id_ex_rs1_addr then
                fwd_a_sel <= "10";
            end if;
            if mem_wb_rd_addr = id_ex_rs2_addr then
                fwd_b_sel <= "10";
            end if;
        end if;

        -----
        -- EX/MEM forwarding (higher priority - overrides MEM/WB
        -- when both stages write to the same destination register)
        -----

        if ex_mem_wb_en = '1' and ex_mem_rd_addr /= "00000" then
            if ex_mem_rd_addr = id_ex_rs1_addr then
                fwd_a_sel <= "01";
            end if;
            if ex_mem_rd_addr = id_ex_rs2_addr then
                fwd_b_sel <= "01";
            end if;
        end if;
    end process;
end architecture;

```

```

        end process fwd_proc;

end architecture behavioral;

```

fetch.vhd

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

library work;
use work.register_pkg.all;
use work.MUX_pkg.all;
use work.ripple_adder_pkg.all;
use work.NOP_factorial_ROM_pkg.all;

entity fetch is
    generic (
        N      : integer := 10;
        M      : integer := 32
    );
    port (
        jump_addr   : in      std_logic_vector(N-1 downto 0);
        pc_select   : in      std_logic;
        stall       : in      std_logic;
        rst         : in      std_logic;
        clk         : in      std_logic;
        decode_addr : out std_logic_vector(N-1 downto 0);
        instruction : out std_logic_vector(M-1 downto 0)
    );
end entity;

architecture component_list of fetch is

    signal new_addr      : std_logic_vector(N-1 downto 0) := (others =>
'0');

```

```

    signal  addr          :    std_logic_vector(N-1 downto 0) := (others =>
'0');
    signal  sum           :    std_logic_vector(N-1 downto 0) := (others => '0');
    signal  C_DUMMY       :    std_logic;

    -- Stall support: MUXed register inputs
    signal  pc_in          :    std_logic_vector(N-1 downto 0);
    signal  dec_addr_in    :    std_logic_vector(N-1 downto 0);
    signal  decode_addr_int :    std_logic_vector(N-1 downto 0);

    constant LSB_ONE      :    std_logic_vector(N-1 downto 0) :=
        (N-2 downto 0 => '0') & '1';
    constant ZERO         :    std_logic := '0';

begin

    -- Stall MUXes: when stall='1', feed back current values to freeze
registers
    pc_in      <= addr              when stall = '1' else new_addr;
    --pc_in <= new_addr; --when stall = '0' else addr;
    dec_addr_in <= decode_addr_int  when stall = '1' else new_addr;
    decode_addr <= decode_addr_int;

    PC_counter :    entity work.reggi
        generic map(
            N => 10
        )
        port map(
            data_in      => pc_in,
            rst           => rst,
            clk           => clk,
            data_out      => addr
        );

    ADDER       :    entity work.ripple_adder
        generic map(
            N => 10
        )
        port map(
            A           => addr,

```

```

        B      => LSB_ONE,
        C_in    => ZERO,
        SUM     => sum,
        C_out   => C_DUMMY
    );

MUXXY      :    entity work.MUX
    generic map(
        N => 10
    )
    port map(
        A      => jump_addr,
        B      => sum,
        S      => pc_select,
        OUTPUT=> new_addr
    );

MUX_REGISTER :    entity work.reggi
    generic map(
        N => 10
    )
    port map(
        data_in => dec_addr_in,
        rst     => rst,
        clk     => clk,
        data_out    => decode_addr_int
    );

--insert IP ROM device with .mif file
IMEM      :    NOP_factorial_ROM
    port map(
        address => addr,
        clock   => clk,
        q       => instruction
    );

end architecture component_list;

-- single dummy instruction to fix bug
-- DEPTH = 1024;

```

```

-- WIDTH = 32;
-- ADDRESS_RADIX = HEX;
-- DATA_RADIX = HEX;
-- CONTENT
-- BEGIN

-- 000 : 04200001; --LW      R1, n(R0)
-- 001 : 0CA10000; --ADD     R5, R1, R0

-- 002 : ACA00013; --BEQZ   R5, 03A
-- 003 : 20A50001; --SUBI   R5, R5, 1
-- 004 : ACA00013; --BEQZ   R5, 03A
-- 005 : 0C610000; --ADD     R3, R1, R0

-- 006 : 0C850000; --ADD     R4, R5, R0
-- 007 : 200D0050; --NOP
-- 008 : 20A50001; --SUBI   R5, R5, 1
-- 009 : ACA00015; --BEQZ   R5, 03F
-- 00A : 0C430000; --ADD     R2, R3, R0

-- 00B : BC00000D; --JAL    028
-- 00C : B4000006; --J      016

-- 00D : 0C631000; --ADD     R3, R3, R2
-- 00E : 20840002; --SUBI   R4, R4, 2
-- 00F : B0800011; --BNEZ   R4, 035
-- 010 : BBE00000; --JR      R31

-- 011 : 10840001; --ADDI   R4, R4, 1
-- 012 : B400000D; --J      028

-- 013 : 10600001; --ADDI   R3, R0, 1
-- 014 : B4000015; --J      03F

-- 015 : 08600000; --SW      f(R0), R3

-- 016 : 04200000; --LW      R1, f(R0)

-- END;

```

```

--trying not to use a dummy instruction
-- DEPTH = 1024;
-- WIDTH = 32;
-- ADDRESS_RADIX = HEX;
-- DATA_RADIX = HEX;
-- CONTENT
-- BEGIN

-- 000 : 04200001; --LW      R1, n(R0)
-- 001 : 0CA10000; --ADD     R5, R1, R0

-- 002 : ACA00013; --BEQZ   R5, 039      (old target 02A)
-- 003 : 20A50001; --SUBI   R5, R5, 1
-- 004 : ACA00013; --BEQZ   R5, 039      (old target 02A)
-- 005 : 0C610000; --ADD     R3, R1, R0

-- 006 : 20A50001; --SUBI   R5, R5, 1
-- 007 : 0C850000; --ADD     R4, R5, R0
-- 008 : 10840001; --ADDI   R4, R4, 1
-- 009 : ACA00015; --BEQZ   R5, 03E      (old target 02F)
-- 00A : 0C430000; --ADD     R2, R3, R0

-- 00B : BC00000D; --JAL     027          (old target 01B)
-- 00C : B4000006; --J       015          (old target 00C)

-- 00D : 0C631000; --ADD     R3, R3, R2
-- 00E : 20840002; --SUBI   R4, R4, 2
-- 00F : B0800011; --BNEZ   R4, 034      (old target 025)
-- 010 : BBE00000; --JR      R31

-- 011 : 10840001; --ADDI   R4, R4, 1
-- 012 : B400000D; --J       027          (old target 01B)

-- 013 : 10600001; --ADDI   R3, R0, 1
-- 014 : B4000015; --J       03E          (old target 02F)

-- 015 : 08600000; --SW     f(R0), R3

-- 016 : 04200000; --LW     R1, f(R0)

```

```

-- END;

--working reordered .mif
-- DEPTH = 1024;
-- WIDTH = 32;
-- ADDRESS_RADIX = HEX;
-- DATA_RADIX = HEX;
-- CONTENT
-- BEGIN

-- 000 : 04200001; --LW      R1, n(R0)
-- 001 : 0CA10000; --ADD     R5, R1, R0

-- 002 : ACA00012; --BEQZ   R5, 039      (old target 02A)
-- 003 : 20A50001; --SUBI   R5, R5, 1
-- 004 : ACA00012; --BEQZ   R5, 039      (old target 02A)
-- 005 : 0C610000; --ADD     R3, R1, R0

-- 006 : 0C850000; --ADD     R4, R5, R0 ; 000011 00100 0
-- 007 : 20A50001; --SUBI   R5, R5, 1
-- 008 : ACA00014; --BEQZ   R5, 03E      (old target 02F)
-- 009 : 0C430000; --ADD     R2, R3, R0

-- 00A : BC00000C; --JAL     027          (old target 01B)
-- 00B : B4000006; --J       015          (old target 00C)

-- 00C : 0C631000; --ADD     R3, R3, R2
-- 00D : 20840002; --SUBI   R4, R4, 2
-- 00E : B0800010; --BNEZ   R4, 034      (old target 025)
-- 00F : BBE00000; --JR      R31

-- 010 : 10840001; --ADDI   R4, R4, 1
-- 011 : B400000C; --J       027          (old target 01B)

-- 012 : 10600001; --ADDI   R3, R0, 1
-- 013 : B4000014; --J       03E          (old target 02F)

-- 014 : 08600000; --SW      f(R0), R3

-- 015 : 04200000; --LW      R1, f(R0)

```

```
-- END;
```

decode.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

library work;
use work.decode_reg_pkg.all;
use work.sign_extend_pkg.all;

entity decode is
    generic (
        FUNC_WIDTH : integer := 6;
        ADDR_WIDTH  : integer := 5;
        DATA_WIDTH  : integer := 32
    );
    port (
        clk          : in  std_logic;
        rst          : in  std_logic;

        -- Hazard control
        stall        : in  std_logic;
        flush        : in  std_logic;

        -- From Fetch
        instruction_in : in  std_logic_vector(31 downto 0);
        pc_inc         : in  std_logic_vector(9  downto 0);

        -- From Writeback
        wb_data        : in  std_logic_vector(DATA_WIDTH-1 downto 0);
        wb_addr        : in  std_logic_vector(ADDR_WIDTH-1 downto 0);
        wb_en          : in  std_logic;
```

```

        -- To Execute
        instruction_out : out std_logic_vector(31 downto 0);
        rs1_data        : out std_logic_vector(DATA_WIDTH-1 downto 0);
        rs2_data        : out std_logic_vector(DATA_WIDTH-1 downto 0);
        sign_ext_imm     : out std_logic_vector(31 downto 0);
        rd_addr_out      : out std_logic_vector(ADDR_WIDTH-1 downto 0);
        pc_inc_out       : out std_logic_vector(9 downto 0)
    );
end entity decode;

architecture structural of decode is
    signal opcode : std_logic_vector(5 downto 0);
    signal rs1_addr, rs2_addr, rd_addr_r : std_logic_vector(4 downto 0);
    signal imm16 : std_logic_vector(15 downto 0);
    signal sign_ext : std_logic_vector(31 downto 0);

    signal reg_dest_sel : std_logic_vector(1 downto 0); -- 00:
-- I-Type(20-16), 01: R-Type(15-11), 10: 31
    -- 00 is also used for instructions that don't write back (Stores,
-- Branches), since wb_en is 0 anyway.

    -- Bubble: insert NOP into pipeline on stall or flush
    signal bubble : std_logic;
    signal decode_rst : std_logic;
    signal instr_next : std_logic_vector(31 downto 0);
    signal pc_next : std_logic_vector(9 downto 0);
    signal imm_next : std_logic_vector(31 downto 0);
    signal reg_instr_out : std_logic_vector(31 downto 0);
    signal reg_sign_ext : std_logic_vector(31 downto 0);
    signal reg_pc_out : std_logic_vector(9 downto 0);

begin
    opcode <= instruction_in(31 downto 26);

    -- When stall or flush, output registers will be cleared (NOP bubble)
    bubble <= flush or stall;
    decode_rst <= rst or bubble;

    --look at (20 downto 16) for all other opcodes except BEQZ BNEZ
    rs1_addr <= instruction_in(20 downto 16) when opcode /= OP_BEQZ and

```

```

opcode /=
OP_BNEZ
--for
Branch instructions look here
--for
address from register
else
instruction_in(25 downto 21);
--look at (15 downto 11) for all other opcodes except SW, JR, JALR
rs2_addr <= instruction_in(15 downto 11) when opcode /= OP_SW and
opcode /=
OP_JR and
opcode /=
OP_JALR
--for SW,
JR, JALR look here
--for
address from register
else
instruction_in(25 downto 21); -- Also acts as RD for I-Type
rd_addr_r <= instruction_in(15 downto 11);
imm16 <= instruction_in(15 downto 0);

instr_next <= instruction_in when stall='0'
else (others => '0');

instruction_out <= reg_instr_out;

instr_reg : entity work.reggi_async
generic map(
N => 32
)
port map(
data_in => instr_next,
rst => decode_rst,
clk => clk,
data_out=> reg_instr_out
);

-- Sign Extender

```

```

sign_extER: sign_extend port map (
    input_data  => imm16,
    output_data => sign_ext
);

sign_ext_imm <= reg_sign_ext;
imm_next <= sign_ext when stall='0'
           else reg_sign_ext;

Sign_reg    : entity work.reggi
generic map(
    N => 32
)
port map(
    data_in => imm_next,
    rst     => decode_rst,
    clk     => clk,
    data_out=> reg_sign_ext
);

-- Register File
reg_file: decode_reg
generic map (DATA_WIDTH => 32, ADDR_WIDTH => 5)
port map (
    clk => clk,
    rst => rst,
    reg_read_addr1 => rs1_addr,
    reg_read_addr2 => rs2_addr,
    reg_write_addr => wb_addr,
    reg_write_data => wb_data,
    reg_write_en   => wb_en,
    reg_read_data1 => rs1_data,
    reg_read_data2 => rs2_data
);

pc_inc_out <= reg_pc_out;
pc_next <= pc_inc when stall='0'
          else reg_pc_out;

-- Pass PC

```

```

PC_register :   entity work.reggi
    generic map(
        N => 10
    )
    port map(
        data_in => pc_next,
        rst      => decode_rst,
        clk      => clk,
        data_out=> reg_pc_out
    );

end architecture structural;

-- Control Logic for RegDst
--process(opcode)
--begin
    --case opcode is
        --when OP_NOP =>
            --reg_dest_sel <= "00";
        --when OP_JAL | OP_JALR =>
            --reg_dest_sel <= "10"; -- Link R31
    -- R-Types
    --when OP_ADD    |
        --OP_ADDU    |
        --OP_SUB     |
        --OP_SUBU    |
        --OP_AND     |
        --OP_OR      |
        --OP_XOR     |
        --OP_SLL     |
        --OP_SRL     |
        --OP_SRA     |
        --OP_SLT     |
        --OP_SLTU    |
        --OP_SGT     |
        --OP_SGTU    |
        --OP_SLE     |
        --OP_SLEU    |
        --OP_SGE     |

```

```

        --OP_SGEU   |
        --OP_SEQ    |
        --OP_SNE    =>
        --reg_dest_sel <= "01"; -- Rd (15-11)

        -- I-Types (default)
--      when others =>
        --      reg_dest_sel <= "00"; -- Rt (20-16)
    --end case;
--end process;

```

execute.vhd

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

library work;
use work.decode_reg_pkg.all;

entity execute is
    generic(
        DATA_WIDTH  : integer := 32;
        PC_WIDTH     : integer := 10
    );
    port (
        clk          : in    std_logic;
        rst          : in    std_logic;

        instruction  : in    std_logic_vector(DATA_WIDTH-1 downto 0);
        pc_inc       : in    std_logic_vector(PC_WIDTH-1 downto 0);

        rs1_data     : in    std_logic_vector(DATA_WIDTH-1 downto 0);
        rs2_data     : in    std_logic_vector(DATA_WIDTH-1 downto
0);
        sign_ext_imm : in    std_logic_vector(DATA_WIDTH-1 downto 0);
        rd_addr_in   : in    std_logic_vector(4 downto 0);

        -- Hazard control

```

```

        flush          : in      std_logic;
        -- Forwarding inputs
        fwd_a_sel       : in      std_logic_vector(1 downto 0);
        fwd_b_sel       : in      std_logic_vector(1 downto 0);
        ex_mem_alu_data: in      std_logic_vector(DATA_WIDTH-1 downto
0);
        mem_wb_data     : in      std_logic_vector(DATA_WIDTH-1 downto
0);

        Branch_en      : out std_logic;
        ALU_result      : out      std_logic_vector(DATA_WIDTH-1 downto 0);
        rs2_data_out    : out std_logic_vector(DATA_WIDTH-1 downto 0);
        instr_out       : out std_logic_vector(DATA_WIDTH-1 downto 0);
        rd_addr_out     : out std_logic_vector(4 downto 0);
        pc_out          : out std_logic_vector(PC_WIDTH-1 downto 0);
        jump_addr       : out std_logic_vector(PC_WIDTH-1 downto 0)
    );
end entity execute;

architecture structural of execute is
    signal opcode        : std_logic_vector(5 downto 0);
    signal reg_opcode    : std_logic_vector(5 downto 0);
    signal ALU_out       : std_logic_vector(DATA_WIDTH-1 downto 0);
    signal branch_reg    : std_logic_vector(0 downto 0);
    signal branch_out    : std_logic_vector(0 downto 0);
    signal q1            : std_logic_vector(DATA_WIDTH-1 downto 0);
    signal q2            : std_logic_vector(DATA_WIDTH-1 downto 0);
    signal ext_pc        : std_logic_vector(DATA_WIDTH-1 downto 0);
    signal reg_ALU       : std_logic_vector(DATA_WIDTH-1 downto 0);
    signal reg_rs2       : std_logic_vector(DATA_WIDTH-1 downto 0);
    signal reg_instr     : std_logic_vector(DATA_WIDTH-1 downto 0);

    -- MUX control signals
    signal mux1_sel      : std_logic; -- '1' = PC (for JAL/JALR), '0' = rs1
    signal mux2_sel      : std_logic; -- '1' = immediate (I-type), '0' = rs2
(R-type)

    -- Forwarding signals
    signal fwd_rs1       : std_logic_vector(DATA_WIDTH-1 downto 0);
    signal fwd_rs2       : std_logic_vector(DATA_WIDTH-1 downto 0);

```

```

    signal flush_rst      :    std_logic;

begin

    ALU_result <= reg_ALU;
    rs2_data_out <= reg_rs2;
    instr_out <= reg_instr;

    -- Combined reset: flush output pipeline registers on branch/jump
    flush_rst <= rst or flush;

    -- Forwarding MUXes: select correct data source for ALU inputs
    -- "00" = no forward (register file), "01" = EX/MEM, "10" = MEM/WB
    fwd_rs1 <= ex_mem_alu_data when fwd_a_sel = "01" else
                mem_wb_data    when fwd_a_sel = "10" else
                rs1_data;

    fwd_rs2 <= ex_mem_alu_data when fwd_b_sel = "01" else
                mem_wb_data    when fwd_b_sel = "10" else
                rs2_data;

    -- Zero-extend PC to DATA_WIDTH
    ext_pc(PC_WIDTH-1 downto 0) <= pc_inc;
    ext_pc(DATA_WIDTH-1 downto PC_WIDTH) <= (others => '0');

    opcode <= instruction(31 downto 26);
    --allows opcode to be synced with jump address and branching
    reg_opcode <= reg_instr(31 downto 26);
    --jump address is reg_ALU(9 downto 0) when opcode isn't JALR or JR
    jump_addr <= reg_ALU(PC_WIDTH-1 downto 0) when reg_opcode /= OP_JALR
AND
    reg_opcode /= OP_JR
                                                                    -- else
look at the botton of reg rs2
                                                                    else
reg_rs2(PC_WIDTH-1 downto 0);

    -- MUX1 Control: Always select rs1_data. PC not needed in ALU for
absolute addressing.

```

```

-- PC is forwarded through pipeline separately for JAL/JALR writeback
to R31 (future lab).
-- For JR/JALR, rs1 contains the jump target, so it MUST pass through
to the ALU.
mux1_sel <= '0';

-- MUX2 Control: Select immediate for I-type instructions, rs2 for
R-type
-- When S='1', MUX outputs A (sign_ext_imm). When S='0', MUX outputs B
(rs2_data).
-- I-type: LW, SW, all Immediate ALU ops, branches, JAL, JALR
-- R-type: register-register ALU ops (ADD, SUB, AND, OR, XOR, shifts,
compares)
mux2_sel <= '0' when (opcode = OP_ADD or opcode = OP_ADDU or
opcode = OP_SUB or opcode = OP_SUBU or
opcode = OP_AND or opcode = OP_OR or
opcode = OP_XOR or
opcode = OP_SLL or opcode = OP_SRL or opcode =
OP_SRA or
opcode = OP_SLT or opcode = OP_SLTU or
opcode = OP_SGT or opcode = OP_SGTU or
opcode = OP_SLE or opcode = OP_SLEU or
opcode = OP_SGE or opcode = OP_SGEU or
opcode = OP_SEQ or opcode = OP_SNE)
else '1';

MUXXY1 : entity work.MUX
generic map(
    N=>32
)
port map(
    A=>ext_pc,
    B=>fwd_rs1,
    S=>mux1_sel,
    OUTPUT=>q1
);

MUXXY2 : entity work.MUX
generic map(
    N=>32

```

```

    )
    port map(
        A=>sign_ext_imm,
        B=>fwd_rs2,
        S=>mux2_sel,
        OUTPUT=>q2
    );

    ALU_inst      :   entity work.ALU
        generic map(
            DATA_WIDTH=>DATA_WIDTH,
            OP_CODE_WIDTH=> 6
        )
        port map(
            data_in1=>q1,
            data_in2=>q2,
            op_sel=>opcode,
            data_out1=>ALU_out
        );

    ALU_out_reg :   entity work.reggi
        generic map(
            N => DATA_WIDTH
        )
        port map(
            data_in=>ALU_out,
            rst=>flush_rst,
            clk=>clk,
            data_out=>reg_ALU
        );

    PC_out_reg : entity work.reggi
        generic map(
            N => PC_WIDTH
        )
        port map(
            data_in=>pc_inc,
            rst=>flush_rst,
            clk=>clk,
            data_out=>pc_out

```

```

);

branch_check_inst    :    entity work.branch_check
    generic map(
        ADDR_WIDTH=>PC_WIDTH
    )
    port map(
        addr_in=>pc_inc,
        rs1=>fwd_rs1,
        opcode=>opcode,
        take_branch=>branch_reg(0)
    );

```

```

branch_check_reg      :    entity work.reggi
    generic map(
        N=>1
    )
    port map(
        data_in=>branch_reg,
        rst=>flush_rst,
        clk=>clk,
        data_out=>branch_out
    );

```

```

Branch_en <= branch_out(0);

```

```

rs2_reg :    entity work.reggi
    generic map(
        N=>DATA_WIDTH
    )
    port map(
        data_in=>fwd_rs2,
        rst=>flush_rst,
        clk=>clk,
        data_out=>reg_rs2
    );

```

```

instr_reg    :    entity work.reggi
    generic map(
        N=>DATA_WIDTH

```

```
)  
port map(  
    data_in=>instruction,  
    rst=>flush_rst,  
    clk=>clk,  
    data_out=>reg_instr  
);  
  
rd_addr_reg : entity work.reggi  
    generic map(  
        N=>5  
    )  
    port map(  
        data_in=>rd_addr_in,  
        rst=>flush_rst,  
        clk=>clk,  
        data_out=>rd_addr_out  
    );  
  
end architecture structural;
```